འཇིགས་མེད་རྣམ་རྒྱལ་བཙན་རྩིག་ཐེག་མཚོའི་རིམ་སྒོང་གྲྭ།

Royal University of Bhutan

Jigme Namgyal Engineering College, Dewathang

DEVICE REPAIR TRACKER

Submitted by

Sherab Pem(05250188)

DIPLOMA IN COMPUTER SYSTEM AND NETWORK

DEPARTMENT OF INFORMATION TECHNOLOGY

DATE 25/9/2026

Contents

1. Introduction 3
2. Technologies Used 3
3. Database Design 3
    3.1 Users Table 3
    3.2 Repairs Table 4
4. Key Features 5
5. Application Walkthrough 5
    5.1 Login Page 5
    5.2 Registration 6
    5.3 Admin Dashboard 7
    5.4 Data Validation 8
    5.5 Search, Sort & Export 9
    5.6 Customer Dashboard 10
    5.7 Logout 11
6. Default Login Accounts 11
7. Conclusion 12

1. Introduction

Device Repair Tracker is a desktop app made with Python (Tkinter) for the interface and SQLite for storage. The idea came from how repair shops usually track repairs on paper, which can easily get lost or messy. So, Device Repair Tracker gives them a digital way to log a device the moment it comes in for repair and follow it all the way through to pick up.

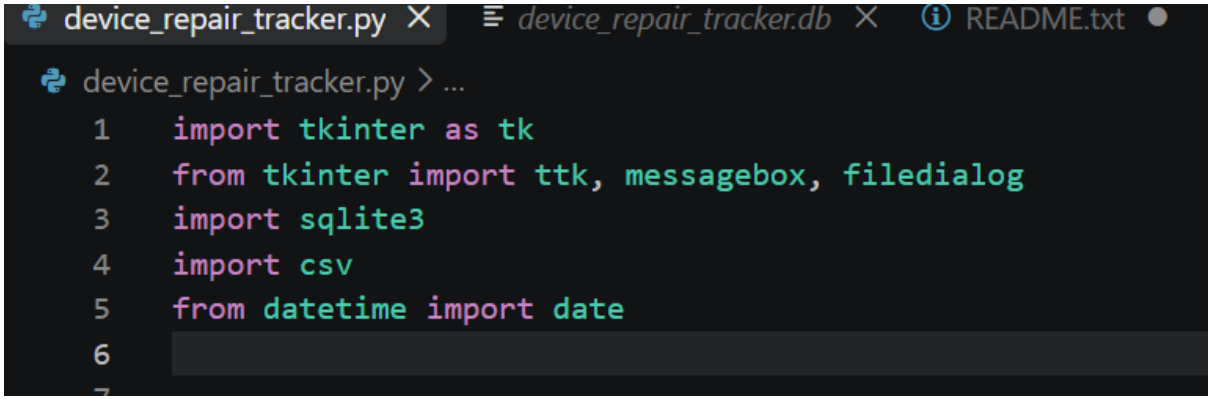
There are two types of users: An Admin (the shop staff) who can see and manage every repair in the system, while a customer can only log in and check the status of their devices. This way, staff keep full control, and customers get an easy way to check on their repair without having to call or visit the shop.

2. Technologies Used

- Python — the programming language used to build the app
- Tkinter / ttk — used to build the windows, buttons, and screens
- SQLite — stores all the data in a simple database file
- CSV module — used to save records as a CSV file
- Visual Studio Code — the program used to write and test the code

3. Database Design

When the app runs for the first time, it automatically creates a database file called device_repair_tracker.db. This database has two tables: one for user accounts, and one for repair records.



3.1 Users Table

This table keeps each user's login details and their role (Admin or Customer). The role decides which screen the person sees after logging in. Every username must be different, so two people cannot share one account.

```
# USERS TABLE
self.cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        role TEXT NOT NULL
    )
""")
```

Figure 1: users table creation script

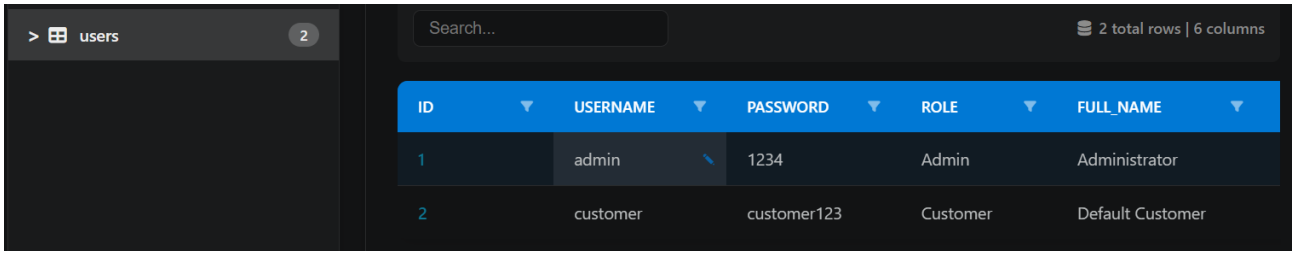


Figure 2: users table data

3.2 Repairs Table

This table stores the details of every repair job , the customer's name and phone number, the device type and model, the problem, the status, the cost, and the date. Each repair is also linked to the customer's username, so the app knows which repairs belong to which customer.

```
82          self.cursor.execute("""
83              CREATE TABLE IF NOT EXISTS repairs (
84                  id INTEGER PRIMARY KEY AUTOINCREMENT,
85                  customer_username TEXT,
86                  customer_name TEXT NOT NULL,
87                  phone TEXT NOT NULL,
88                  device_type TEXT NOT NULL,
89                  device_model TEXT NOT NULL,
90                  problem TEXT NOT NULL,
91                  repair_status TEXT NOT NULL,
92                  repair_cost REAL NOT NULL,
93                  repair_date TEXT NOT NULL
94              )
95          """)
96
97
```

Figure 3: repairs table creation script

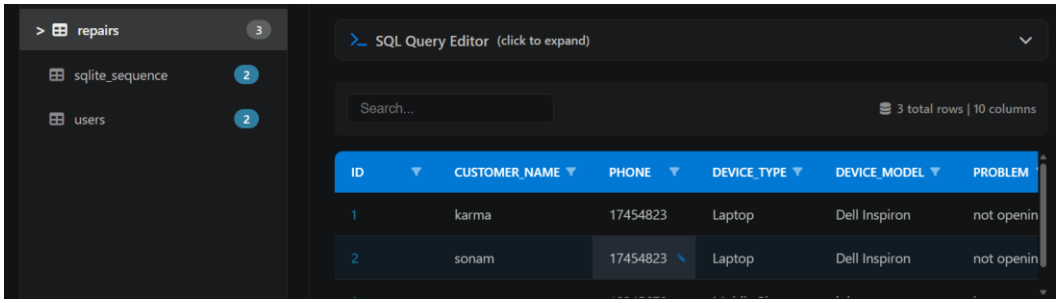


Figure 4: repairs table data

4. Key Features

- Separate login for Admin and Customer
- New users can create their own account
- Admin can add, edit, delete, and clear repair records
- Search box to quickly find any record
- Click a column to sort the records
- Export records to a CSV file
- Customers can see their own repairs
- Warning messages stop missing or incorrect data from being saved
- Safe logout that clears the session

5. Application Walkthrough

This section explains each screen in the app and what it is used for.

5.1 Login Page

This is the first screen. The user picks Admin or Customer, then they enters a username and password to sign in. There is also an option to show the typed password.

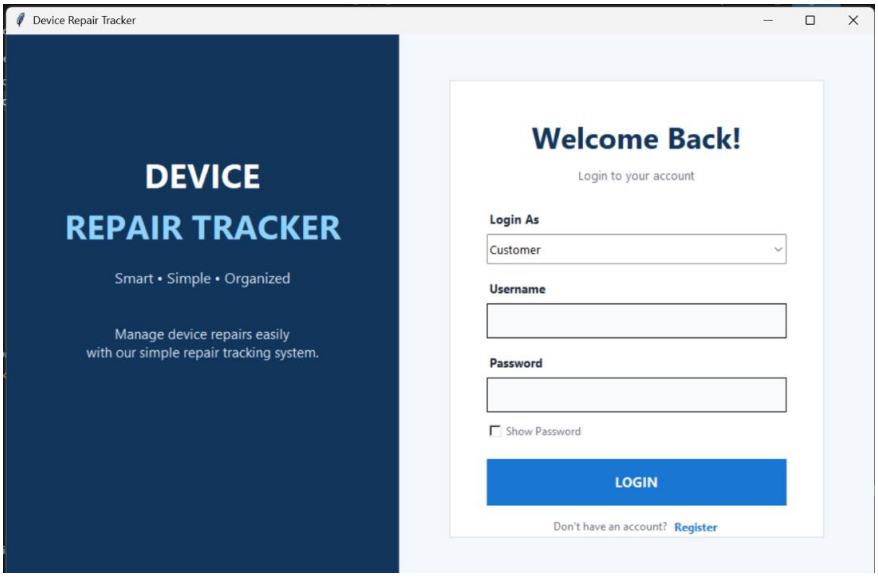


Figure 5: Login page

5.2 Registration

This screen lets a new user create an account. It asks for name, phone number, role, username, and password, and checks that everything is filled in correctly before saving.

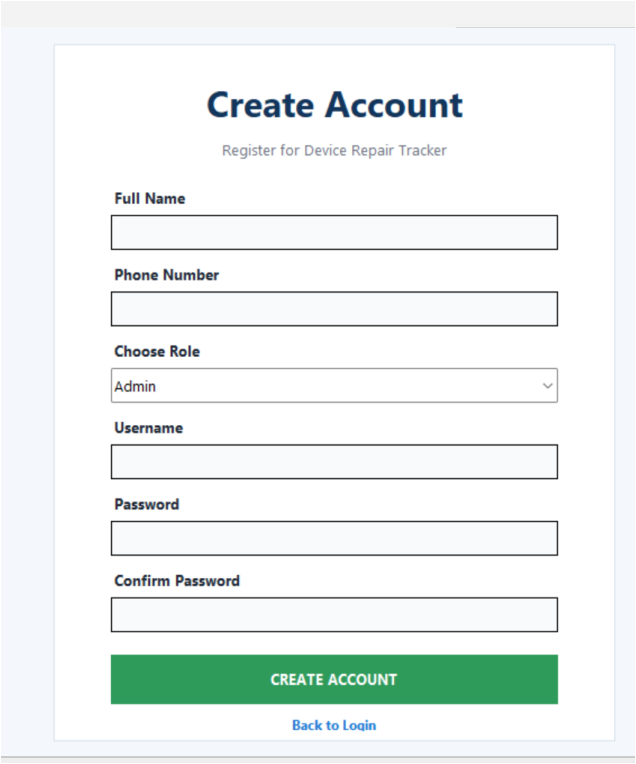
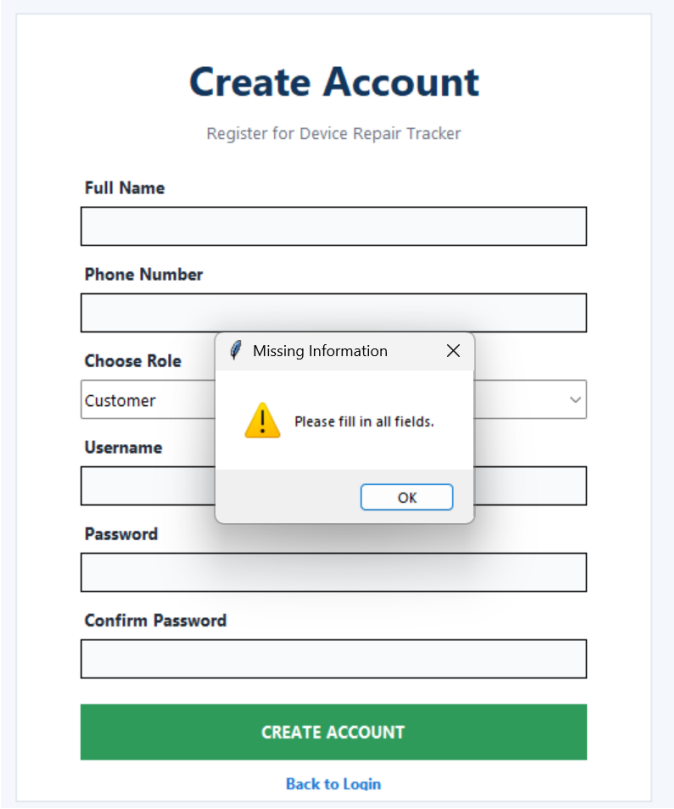


Figure 6: Account registration form



5.3 Admin Dashboard

This is the main screen for Admin. It has a form to add repair details, a search box, and a table showing all repairs on record. For Device Type and Repair Status, I used dropdown menus instead of free text boxes, mainly so the data stays consistent and doesn't end up with typos like "laptp" or "pending " with an extra space.

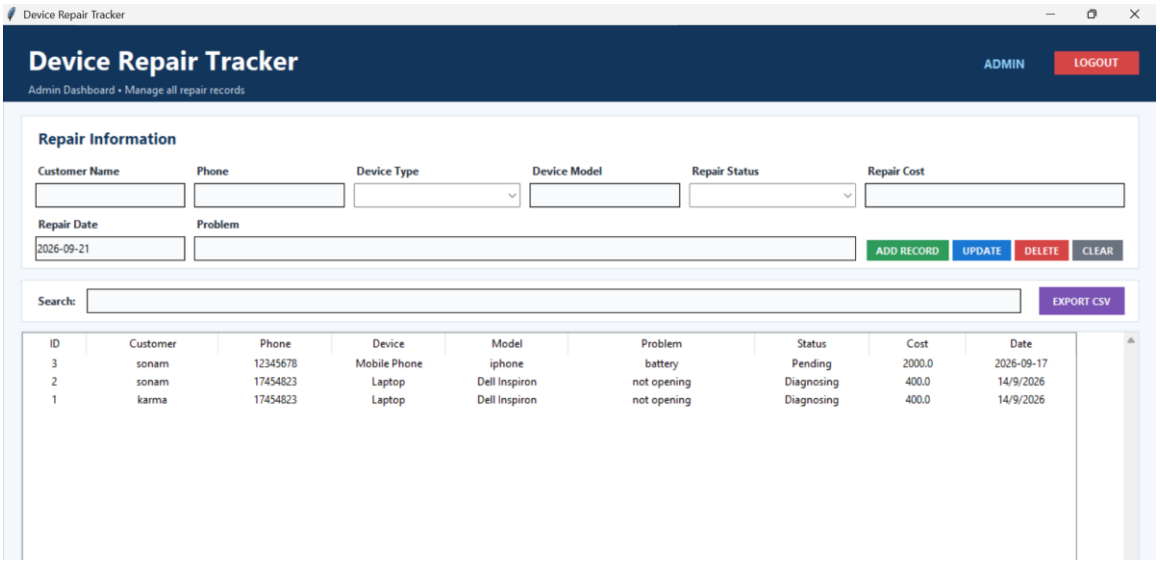


Figure 7: Admin dashboard with repair records

Shows: the repair entry form, the action buttons (Add, Update, Delete, Clear), and the full list of repair records below.

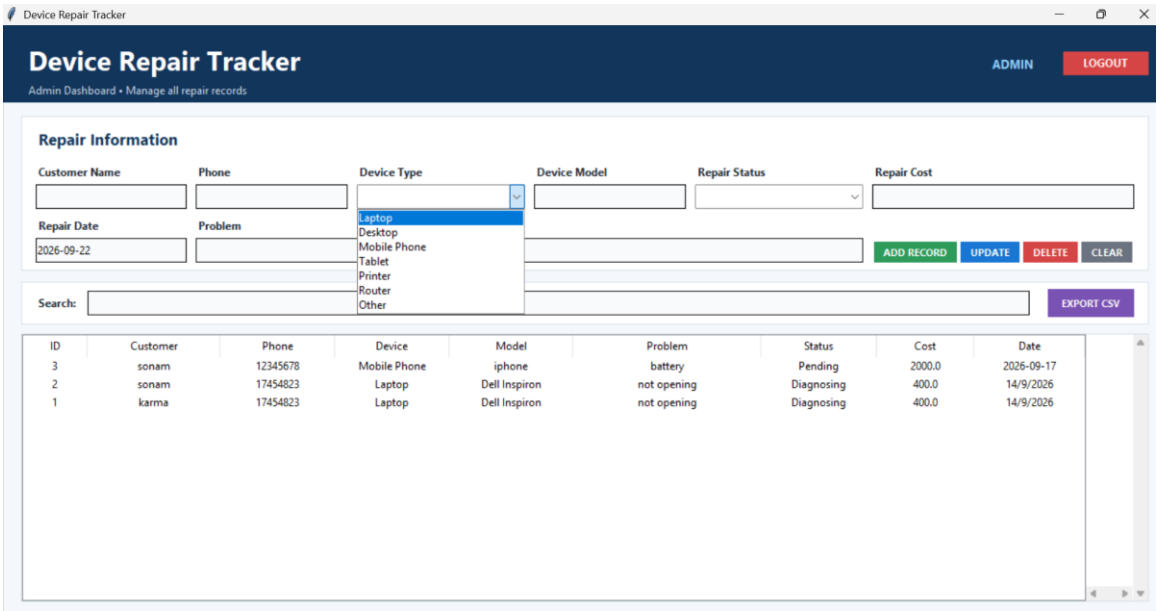


Figure 8: Device Type selection

Shows: the Device Type dropdown open, listing Laptop, Desktop, Mobile Phone, Tablet, Printer, Router, and Other.

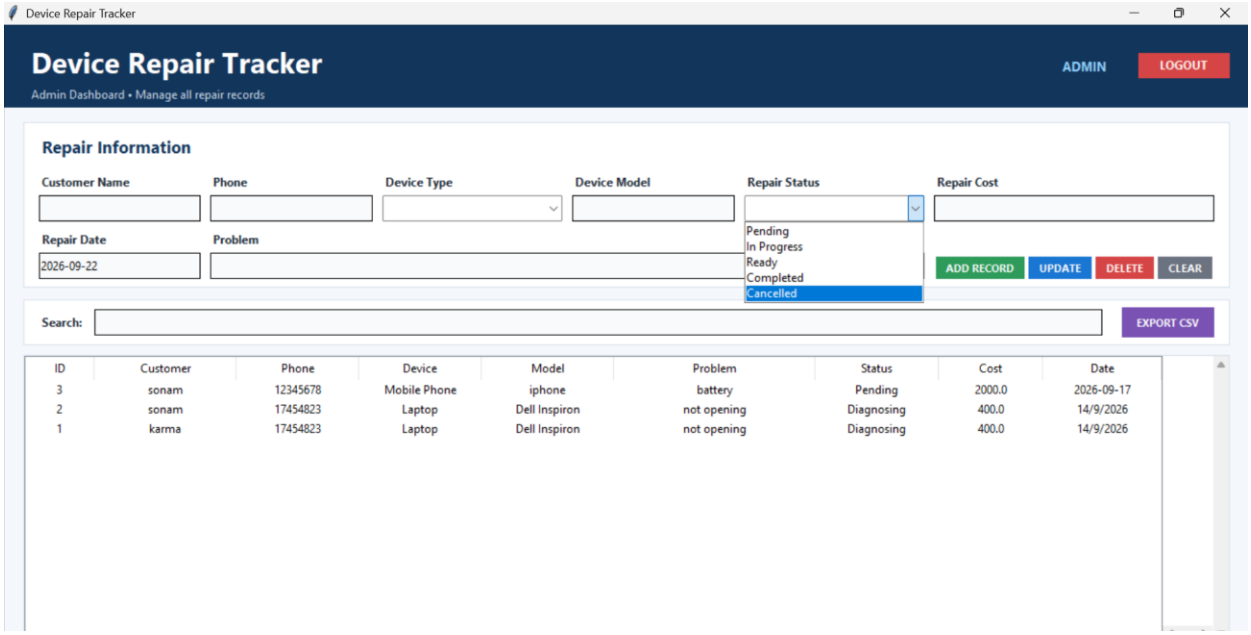


Figure 9: Repair Status selection

Shows: the Repair Status dropdown open, listing Pending, In Progress, Ready, Completed, and Cancelled.

5.4 Data Validation

These are warning pop-ups. I added a few checks so the Admin can't accidentally break the data. If a required field like customer name is left blank, the app stops and shows a warning instead of saving an incomplete record. The same thing happens if Update or Delete is clicked without a record selected first the app simply asks the Admin to pick one .

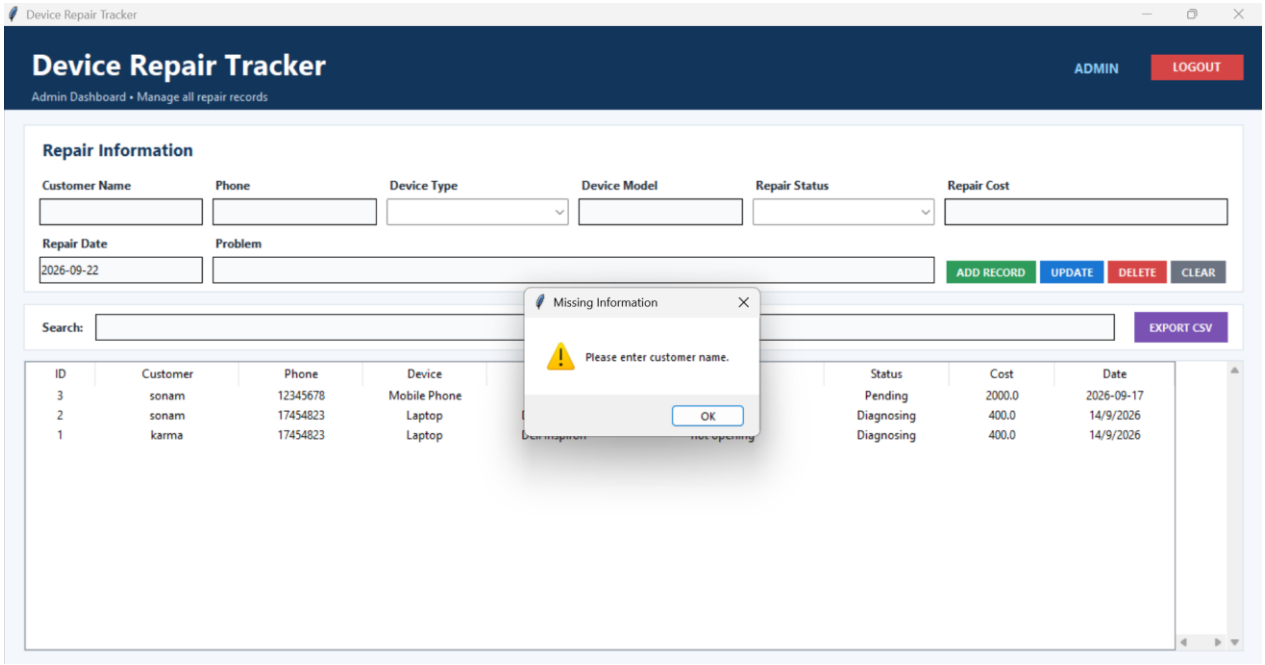


Figure 10: Missing information warning

Shows: a pop-up warning that appears when a required field, like customer name, is left empty.

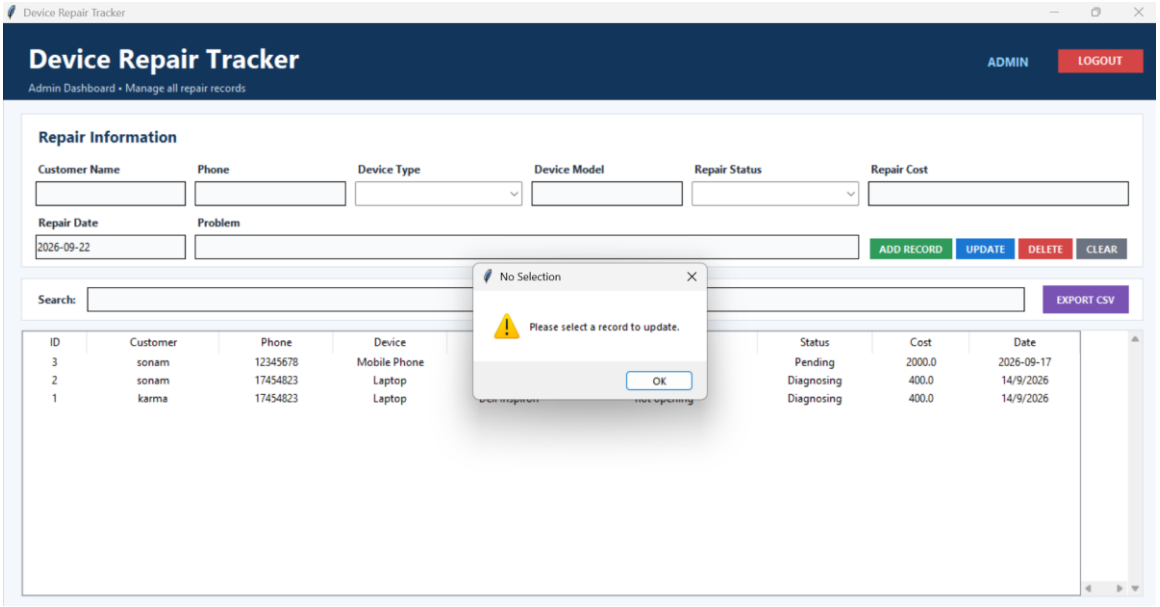


Figure 11: No record selected for update

Shows: a pop-up warning that appears when Update is clicked without first selecting a record.

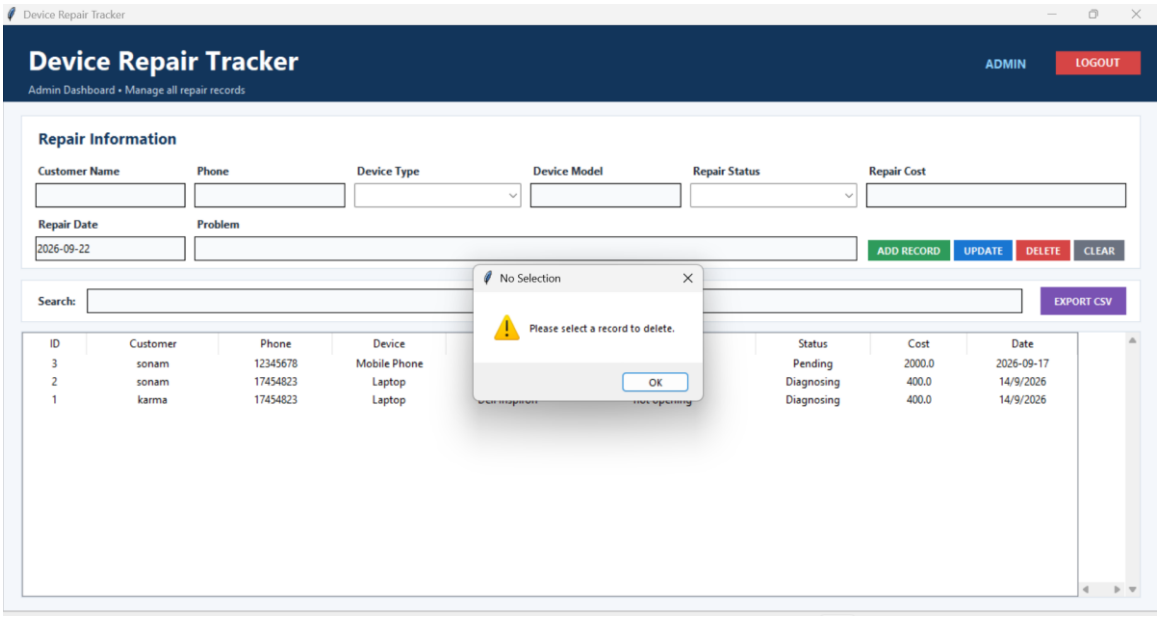


Figure 12: No record selected for delete

Shows: a pop-up warning that appears when Delete is clicked without first selecting a record.

5.5 Search, Sort & Export

The search box filters records as you type. Clicking a column heading sorts the table by that column. The EXPORT CSV button saves the records shown on screen into a CSV file, which the Admin can then open in Excel or attach to an email.

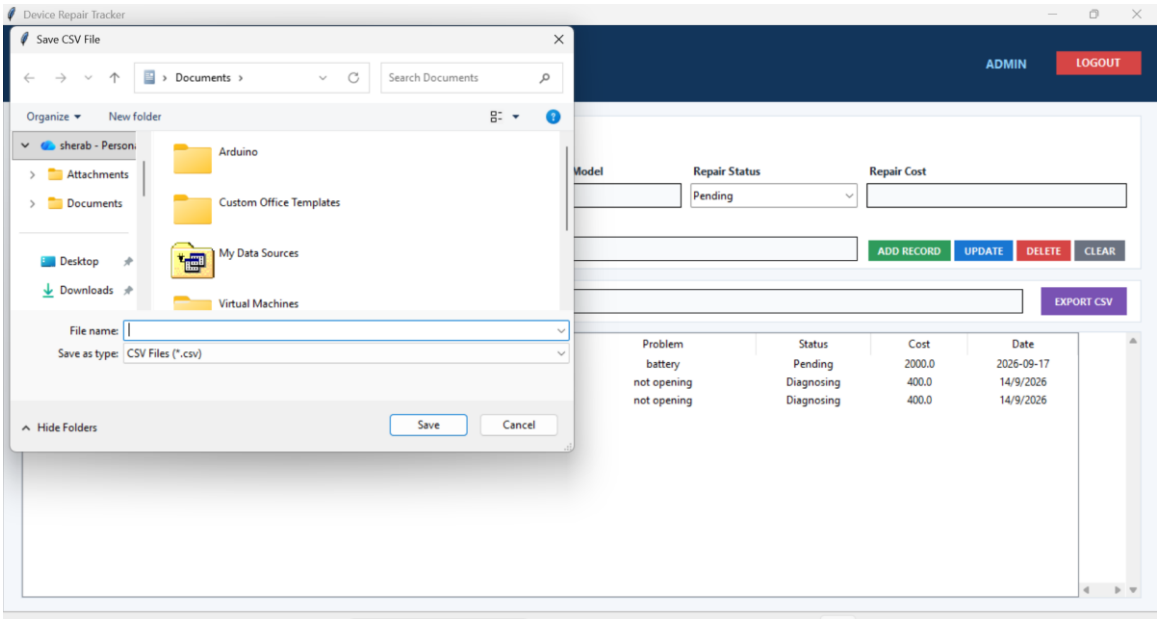


Figure 13: Exporting records to CSV

Shows: the Save File window that opens after clicking EXPORT CSV, where the user names the file and saves it.

5.6 Customer Dashboard

This is the screen a customer lands on after logging in. It's much simpler than the admin view on, it just shows a quick summary of how many repairs they have, how many are In Progress, and how many are Ready, followed by a table of their devices.

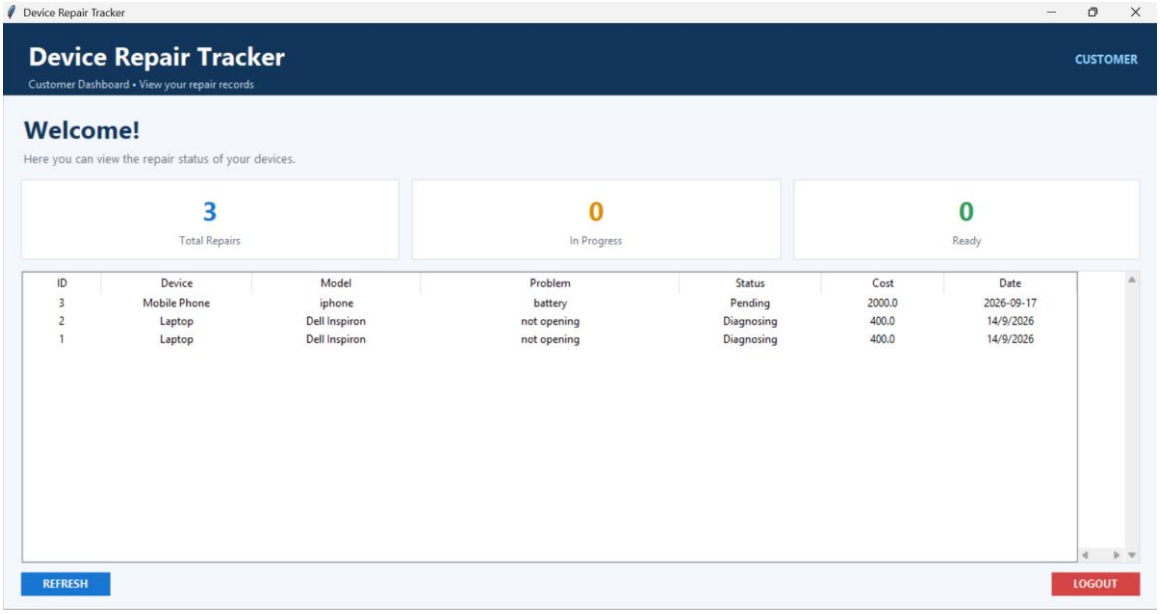


Figure 14: Customer dashboard

Shows: the summary cards (Total Repairs, In Progress, Ready) and a table listing only the logged-in customer's own devices.

5.7 Logout

Clicking Logout asks for confirmation first. Once confirmed, the session ends and the app goes back to the Login Page.

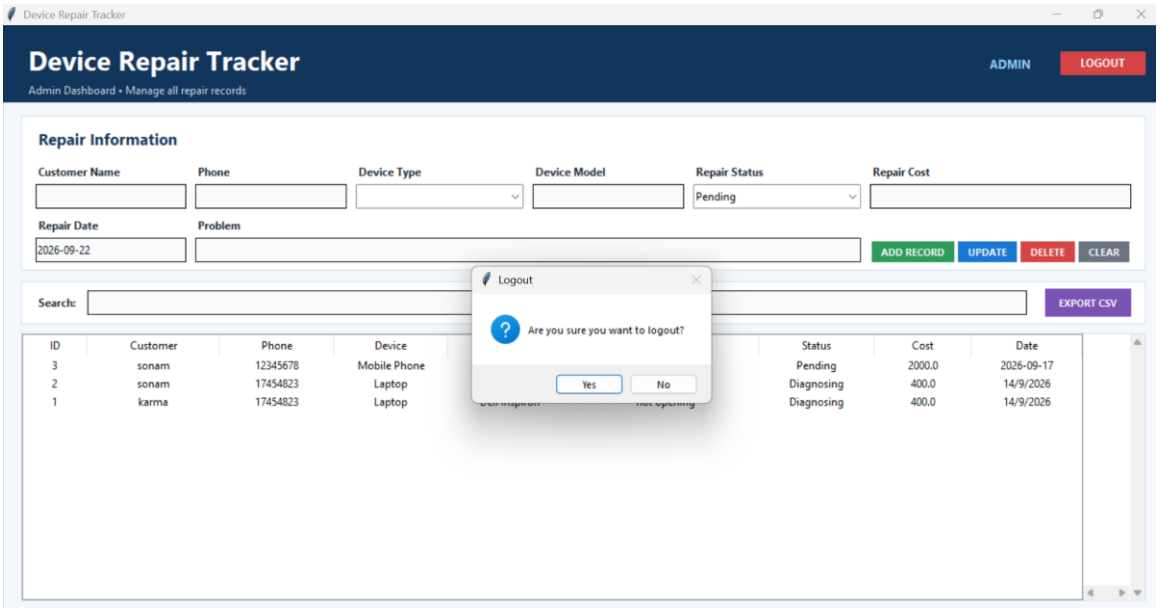


Figure 15: Logout confirmation

Shows: the Yes/No confirmation pop-up that appears when the LOGOUT button is clicked.

6. Default Login Accounts

These two accounts are already set up, so you can log in right away for testing:

Role	Username / Password
Admin	admin / 1234
Customer	customer / customer123

7. Conclusion

Device Repair Tracker is a simple but complete example of a desktop app built with Python and Tkinter. It covers login, user roles, adding and editing records, searching, sorting, and exporting data all the basic parts of a real business tool. It also keeps customer information private while giving shop staff full control, making it a good example of how a small, practical application comes together.